COMPUTING LAB - II
ADVANCED SYSTEM SOFTWARE LAB - CSE 5261
MTech - CSE
Week - 4 Lexical Analyzer

Name : Rajat Jain
Reg No : 200913010

Lab Exercises:

1) Write a python program to design a lexical analyzer for a simple C program to create a token each time and return, which includes row number, column number and token type. The tokens to be identified are arithmetic operators, relational operators, logical operators, special symbols, keywords, numerical constants, string literals and identifiers. Also, it should ignore all the tokens when encountered inside single line or multi-line comment or inside string literal. Pre-processor directive should also be stripped.

Code

```
from ply import lex
import re


tokens = [
    # Literals (identifier, integer constant, float constant, string constant, char
const)
    'ID', 'TYPEID', 'INTEGER', 'FLOAT', 'STRING',
'CHARACTER','COMMENT',

    # Operators (+,-,*,/,%,|,&,~,^,<<,>>, ||, &&, !, <, <=, >, >=, ==, !=)
    'PLUS', 'MINUS', 'TIMES', 'DIVIDE', 'MODULO',
    'OR', 'AND', 'NOT', 'XOR', 'LSHIFT', 'RSHIFT',
    'LOR', 'LAND', 'LNOT',
    'LT', 'LE', 'GT', 'GE', 'EQ', 'NE',

    # Assignment (=, *=, /=, %=, +=, -=, <<=, >>=, &=, ^=, |=)
    'EQUALS', 'TIMESEQUAL', 'DIVEQUAL', 'MODEQUAL', 'PLUSEQUAL',
'MINUSEQUAL',
    'LSHIFTEQUAL','RSHIFTEQUAL', 'ANDEQUAL', 'XOREQUAL',
'OREQUAL',
```

```
    # Increment/decrement (++,--)
    'INCREMENT', 'DECREMENT',

    # Structure dereference (->)
    'ARROW',

    # Ternary operator (?)
    'TERNARY',

    # Delimeters ( ) [ ] { } , . ; :
    'LPAREN', 'RPAREN',
    'LBRACKET', 'RBRACKET',
    'LBRACE', 'RBRACE',
    'COMMA', 'PERIOD', 'SEMI', 'COLON',

    # Ellipsis (...)
    'ELLIPSIS',

    # Keywords
    'KEYWORD',
]

# Operators
t_PLUS             = r'\+'
t_MINUS            = r'-'
t_TIMES            = r'\*'
t_DIVIDE           = r'/'
t_MODULO           = r'%'
t_OR               = r'\|'
t_AND              = r'&'
t_NOT              = r'~'
t_XOR              = r'\^'
t_LSHIFT           = r'<<'
t_RSHIFT           = r'>>'
t_LOR              = r'\|\|'
t_LAND             = r'&&'
t_LNOT             = r'!'
t_LT               = r'<'
t_GT               = r'>'
t_LE               = r'<='
t_GE               = r'>='
t_EQ               = r'=='
```

```
t_NE            = r'!='

# Assignment operators

t_EQUALS        = r'='
t_TIMESEQUAL    = r'\*='
t_DIVEQUAL      = r'/='
t_MODEQUAL      = r'%='
t_PLUSEQUAL     = r'\+='
t_MINUSEQUAL    = r'-='
t_LSHIFTEQUAL   = r'<<='
t_RSHIFTEQUAL   = r'>>='
t_ANDEQUAL      = r'&='
t_OREQUAL       = r'\|='
t_XOREQUAL      = r'\^='

# Increment/decrement
t_INCREMENT     = r'\+\+'
t_DECREMENT     = r'--'

# ->
t_ARROW         = r'->'

# ?
t_TERNARY       = r'\?'

# Delimeters
t_LPAREN        = r'\('
t_RPAREN        = r'\)'
t_LBRACKET      = r'\['
t_RBRACKET      = r'\]'
t_LBRACE        = r'\{'
t_RBRACE        = r'\}'
t_COMMA         = r','
t_PERIOD        = r'\.'
t_SEMI          = r';'
t_COLON         = r':'
t_ELLIPSIS      = r'\.\.\.'

# Identifiers
t_ID = r'[A-Za-z_][A-Za-z0-9_]*'
```

```
# Integer literal
t_INTEGER = r'\d+([uU]|[lL]|[uU][lL]|[lL][uU])?'

# Floating literal
t_FLOAT = r'((\d+)(\.\d+)(e(\+|-)?(\d+))? | (\d+)e(\+|-)?(\d+))([lL]|[fF])?'

# String literal
t_STRING = r'\"([^\\\n]|(\\.))*?\"'

# Character constant 'c' or L'c'
t_CHARACTER = r'(L)?\'([^\\\n]|(\\.))*?\'

key_words = ['auto',        'break',        'case', 'char', 'do$',
'const',        'continue',    'default',
'double',       'else',  'enum',        'extern',
'float', 'for',   'goto', 'if',
'int',    'long', 'register',      'return',
'short',        'signed',       'sizeof',        'static',
'struct',       'switch',       'typedef',      'union',
'unsigned',    'void', 'volatile',      'while']

t_KEYWORD = r'|'.join(key_words)

t_COMMENT = r'/\*(.|\n)*?\*/|//.*\n'

# Compute column.
 #     input is the input text string
 #     token is a token instance
def find_column(input, token):
    line_start = input.rfind('\n', 0, token.lexpos) + 1
    return (token.lexpos - line_start) + 1


def t_newline(t):
  r'\n+'
  t.lexer.lineno += len(t.value)




t_ignore = ' \t'
```

```
def t_error(t):
  print(f"Illegal character {t.value[0]}")
  t.lexer.skip(1)

lex.lex()

lines = []
with open('cprogram.c','r') as file:
	for line in file:
		if not line.startswith('#'):
			lines.append(line)

data = ''.join(lines)

print(data)

generated_tokens = []

lex.input(data)
#Tokenize
while 1:
	tok = lex.token()
	if not tok: break
	print(tok)
	print(f"Column : {find_column(data,tok)} Row : {tok.lineno} Type :
{tok.type} Value : {tok.value}")
	token = f'<{tok.lineno},{find_column(data,tok)},{tok.value},{tok.type}>'
	generated_tokens.append(token)
output = '\n'.join(generated_tokens)
print(output)
with open('output.txt','w') as file:
	file.write(output)
	file.write(f'\nTotal number of tokens generated =
{len(generated_tokens)}')
```

Output

cprogram - Notepad

File Edit Format View Help

```
#include<stdio.h>
#define MAX 20
int main()
{
int a,b; /* declaration */
a=b; /* assignment */
printf("The number is ",a); /* print statement */
}
```

output - Notepad

File Edit Format View Help

```
<1,1,int,KEYWORD>
<1,5,main,ID>
<1,9,(,LPAREN>
<1,10,),RPAREN>
<2,1,{,LBRACE>
<3,1,int,KEYWORD>
<3,5,a,ID>
<3,6,,,COMMA>
<3,7,b,ID>
<3,8,;,SEMI>
<3,10,/* declaration */,COMMENT>
<4,1,a,ID>
<4,2,=,EQUALS>
<4,3,b,ID>
<4,4,;,SEMI>
<4,6,/* assignment */,COMMENT>
<5,1,printf,ID>
<5,7,(,LPAREN>
<5,8,"The number is ",STRING>
<5,24,,,COMMA>
<5,25,a,ID>
<5,26,),RPAREN>
<5,27,;,SEMI>
<5,29,/* print statement */,COMMENT>
<6,1,},RBRACE>
Total number of tokens generated = 25
```

2) Write a python program to design a simple arithmetic calculator.

Code

```
from ply import lex

tokens = (   'NUMBER',   'PLUS', 'SPACE',   'MINUS',   'TIMES',   'DIVIDE',
'LPAREN',   'RPAREN',)
#Regular expression rules for simple tokens
t_PLUS    = r'\+'
t_MINUS   = r'-'
t_TIMES   = r'\*'
t_DIVIDE  = r'/'
t_LPAREN  = r'\('
t_RPAREN  = r'\)'
t_SPACE = r'\ '

def t_NUMBER(t):
  r'\d+'
  try:
    t.value = int(t.value)
  except ValueError:
    print(f"Line {t.lineno}: Number {t.value} is too large!")
    t.value = 0
  return t

# Compute column.
 #     input is the input text string
 #     token is a token instance
def find_column(input, token):
    line_start = input.rfind('\n', 0, token.lexpos) + 1
    return (token.lexpos - line_start) + 1


def t_newline(t):
  r'\n+'
  t.lexer.lineno += len(t.value)

t_ignore = '\t'

def t_error(t):
  print(f"Illegal character {t.value[0]}")
  t.lexer.skip(1)
```

```
lex.lex()

# Test it out
data = '''
3+4
10*2
5/6
7-2
'''

lex.input(data)
#Tokenize
while 1:
	tok = lex.token()
	if not tok: break
	#print(tok)
	print(f'<{tok.lineno},{find_column(data,tok)},{tok.value},{tok.type}>\n'
)
```

<u>Output</u>

```
<2,1,3,NUMBER>

<2,2,+,PLUS>

<2,3,4,NUMBER>

<3,1,10,NUMBER>

<3,3,*,TIMES>

<3,4,2,NUMBER>

<4,1,5,NUMBER>

<4,2,/,DIVIDE>

<4,3,6,NUMBER>

<5,1,7,NUMBER>

<5,2,-,MINUS>

<5,3,2,NUMBER>
```